

Tareas 04 y 05: Sistema Multi-Agente con RAG, A2A, MCP y Memoria

Informe Preliminar — Primera Entrega

Fabrizio Mena Mejía
Instituto Tecnológico de Costa Rica
fabriciomena11@estudiantec.cr

Anthony Fuentes Calvo
Instituto Tecnológico de Costa Rica
anthonyfuentescalvo@estudiantec.cr

Fernando Sánchez Hidalgo
Instituto Tecnológico de Costa Rica
Fer.sanchez@estudiantec.cr

Abstract—Este informe documenta el prototipo funcional desarrollado como primera entrega del proyecto de sistema multi-agente para el curso de Inteligencia Artificial del Instituto Tecnológico de Costa Rica. El sistema implementado integra un agente orquestador basado en GPT-4o, una canalización RAG (*Retrieval-Augmented Generation*) sobre los apuntes del curso almacenados en ChromaDB, un agente de búsqueda web mediante la API de Tavily, un agente resumidor y observabilidad completa a través de Langfuse. Se describe la arquitectura inicial, las decisiones técnicas tomadas, los problemas encontrados durante el desarrollo y el plan de trabajo para incorporar los componentes restantes en la segunda entrega.

I. INTRODUCCIÓN

El objetivo del proyecto es construir un sistema multi-agente capaz de responder preguntas académicas apoyándose en los apuntes del curso de Inteligencia Artificial, fuentes externas en internet y, en la segunda etapa, en una base de datos transaccional ficticia. El sistema debe combinar técnicas de RAG, orquestación de agentes, memoria conversacional y observabilidad de trazas.

Esta primera entrega corresponde al prototipo funcional. Se implementaron el agente orquestador, la canalización RAG con ChromaDB, el agente de búsqueda web, el agente resumidor y la observabilidad con Langfuse, además de una interfaz web mínima con Streamlit. En las secciones siguientes se documenta la arquitectura adoptada, las decisiones técnicas relevantes, los problemas encontrados y el plan para la segunda entrega.

II. DESCRIPCIÓN GENERAL DE LA ARQUITECTURA

La arquitectura del prototipo sigue un patrón de orquestación centralizada. Un agente principal (*Orchestrator*) recibe cada pregunta del usuario, decide qué herramienta invocar y sintetiza la respuesta final. Los componentes principales son:

- **Base documental:** apuntes del curso en formato PDF, preprocesados y almacenados en ChromaDB como vectores de embeddings.
- **Agente Orquestador:** modelo gpt-4o, encargado de seleccionar herramientas y construir la respuesta final.
- **Agente RAG:** modelo gpt-4o-mini, genera respuestas a partir de los fragmentos recuperados de la base vectorial.

- **Agente Web Search:** modelo gpt-4o-mini con la API de Tavily para búsquedas externas.
- **Agente Resumidor:** modelo gpt-4o-mini, genera resúmenes de semanas específicas usando búsqueda vectorial filtrada por metadato.
- **Observabilidad:** Langfuse registra trazas de cada ejecución, incluyendo spans de recuperación, generación y llamadas entre agentes.
- **Interfaz web:** aplicación Streamlit que expone el agente al usuario.

III. PREPROCESAMIENTO TEXTUAL Y BASE DOCUMENTAL

A. Extracción y limpieza

Los documentos del dataset provienen de apuntes elaborados por estudiantes del curso, nombrados con la convención <semana>_SEMANA_AI_<fecha>_<número>.pdf. El proceso de extracción utilizó la librería pypdf para leer cada página. Sobre el texto crudo se aplicaron las siguientes operaciones de limpieza:

- Eliminación de caracteres de control y saltos de línea redundantes.
- Normalización Unicode para corregir tildes y caracteres especiales del español.
- Extracción automática de metadatos a partir del nombre del archivo mediante expresiones regulares: número de semana, fecha de elaboración y número de documento.

B. Metadatos extraídos

Cada fragmento almacenado en la base vectorial incluye los siguientes metadatos:

TABLE I
METADATOS ALMACENADOS POR FRAGMENTO

Campo	Descripción
semana	Número de semana del curso (entero)
fecha	Fecha del apunte (YYYYMMDD)
nombre_archivo	Nombre original del PDF
chunk_index	Índice del fragmento dentro del documento

El metadato *semana* resulta especialmente útil para el agente resumidor, que filtra directamente la colección Chro-

maDB con `where={"semana": week}` antes de ejecutar la consulta vectorial.

C. Segmentación

La estrategia de segmentación utilizada en el prototipo es `RecursiveCharacterTextSplitter` de `LangChain` con los siguientes parámetros:

- `chunk_size` = 500 caracteres.
- `chunk_overlap` = 100 caracteres.

Esta configuración fue elegida como punto de partida por ser balanceada entre granularidad semántica y costo de embeddings. En la segunda entrega se implementará y comparará una segunda estrategia de segmentación (por ejemplo, segmentación por párrafo o por oración) para evaluar cuál produce mejores resultados de recuperación.

IV. EMBEDDINGS Y BASE VECTORIAL

A. Modelo de embeddings

Se utilizó el modelo `text-embedding-3-small` de OpenAI para generar los vectores de los fragmentos. La selección se justifica por los siguientes factores:

- **Disponibilidad:** accesible directamente a través de la API de OpenAI, sin infraestructura adicional.
- **Costo:** es el modelo de embeddings de menor costo en el catálogo de OpenAI.
- **Dimensión:** produce vectores de 1536 dimensiones, compatibles con ChromaDB sin configuración adicional.
- **Soporte multilingüe:** maneja correctamente texto en español.

B. Base vectorial

Se utilizó ChromaDB con almacenamiento persistente en disco (`PersistentClient`). La colección se denomina `apuntes_ai` y los vectores se generan una única vez; en ejecuciones posteriores se reutilizan si la colección ya existe. El umbral de distancia coseno para filtrar fragmentos poco relevantes se fijó en `MAX_DISTANCE = 0.6`.

V. DISEÑO DE AGENTES Y RESPONSABILIDADES

A. Agente Orquestador

El orquestador utiliza el modelo `gpt-4o` con *function calling*. Recibe la pregunta del usuario y decide, sin intervención manual, qué herramienta invocar. El *system prompt* define tres herramientas disponibles:

- `consultar_apuntes`: invoca el pipeline RAG.
- `resumir_semana`: genera un resumen semanal filtrado por metadato.
- `buscar_en_web`: ejecuta una búsqueda Tavily cuando la información no se encuentra en los apuntes o el usuario lo solicita explícitamente.

Una regla importante en el prompt es el *fallback* automático: si `consultar_apuntes` retorna la cadena “No encontré información suficiente”, el orquestador escala automáticamente a `buscar_en_web` sin pedir confirmación al usuario.

B. Agente RAG

Implementado en `rag.py`, este agente recupera los $k = 5$ fragmentos más relevantes de ChromaDB, filtra por distancia coseno y construye un contexto que envía al modelo `gpt-4o-mini`. El prompt instruye al modelo a responder únicamente desde el contexto recuperado, reduciendo alucinaciones.

C. Agente Web Search

Implementado en `websearch.py`, este agente llama a la API de Tavily con hasta cinco resultados, construye un contexto con título, URL y fragmento de cada resultado, y genera la respuesta con `gpt-4o-mini`.

D. Agente Resumidor

Implementado dentro de `agent.py` como la función `summarize_week`. Realiza una búsqueda vectorial en ChromaDB filtrada por el metadato `semana` y genera un resumen con el modelo `gpt-4o-mini`.

E. Contratos de agentes

A continuación se muestra el contrato mínimo del Agente RAG como ejemplo:

```
{
  "agent_name": "NotesRAGAgent",
  "description": "Recupera informacion
    desde los apuntes del curso.",
  "skills": ["search_notes",
    "answer_from_notes"],
  "allowed_tools": ["rag_tool"],
  "restrictions": [
    "Responder solo desde el contexto.",
    "No inventar fuentes.",
    "No usar busqueda web."
  ]
}
```

VI. OBSERVABILIDAD CON LANGFUSE

Se integró Langfuse mediante el SDK oficial de Python (`langfuse.openai`). Cada ejecución del agente crea un *span* raíz llamado `agent_run` que anida los siguientes sub-spans:

- `agent_decision`: captura la llamada de *function calling* al orquestador.
- `tool_calls`: registra la ejecución de cada herramienta invocada.
- `rag_pipeline` → `retrieve_documents` → `generate_answer`: trazabilidad interna del pipeline RAG.
- `websearch_pipeline` → `web_search_retrieve` → `websearch_generate_answer`: trazabilidad del agente web.
- `agent_final_response`: respuesta sintetizada por el orquestador.

Adicionalmente, el pipeline RAG registra tres métricas de calidad por traza: `context_relevance` (calculada como $1 - \bar{d}$ donde $\bar{d}$ es la distancia coseno promedio), `answer_correctness` y `groundedness`.

VII. INTERFAZ WEB

La interfaz se implementó con Streamlit (app.py). Permite al usuario ingresar una pregunta en un campo de texto y obtener la respuesta generada por el agente junto con los documentos recuperados y sus metadatos desplegados en secciones expandibles. La interfaz es intencionalmente minimalista para esta primera entrega; en la segunda entrega se ampliará para soportar memoria conversacional visible.

VIII. PROBLEMAS ENCONTRADOS

Durante el desarrollo del prototipo se encontraron los siguientes problemas:

- 1) **Calidad de los PDFs:** varios apuntes presentaron texto con codificación irregular o caracteres corruptos tras la extracción con pypdf. Fue necesario aplicar normalización Unicode y eliminar caracteres no imprimibles antes de segmentar.
- 2) **Extracción de metadatos desde el nombre de archivo:** algunos archivos no seguían la convención de nomenclatura esperada (por ejemplo, 13_SEMANA_AI_2018191805_1.pdf), lo que hizo que la expresión regular retornara None para los campos de semana y fecha. Se implementó un manejo de casos por defecto, pero estos documentos se almacenan sin metadato de semana y no pueden ser filtrados por el agente resumidor.
- 3) **Límites de la API de Tavily:** durante pruebas intensivas, la cuenta de Tavily en capa gratuita alcanzó el límite de peticiones, interrumpiendo el flujo del agente web. Se implementó manejo de excepciones que retorna una lista vacía y el orquestador informa al usuario que no se encontraron resultados.
- 4) **Sincronización del contexto de Langfuse:** al ejecutar el agente en múltiples hilos (por la naturaleza de Streamlit), las trazas ocasionalmente se anidaban de forma incorrecta. Se resolvió llamando a `lf.flush()` al final de cada ejecución de agente para garantizar el envío inmediato.
- 5) **Costo de generación de embeddings:** el proceso inicial de generación de embeddings para todo el dataset tomó varios minutos y generó costos no triviales en la API de OpenAI, consumiendo alrededor de 914k tokens el primer día ya que se regeneraron varias veces debido a malos formateos en el texto de los datos adquiridos de los pdf. Se agregó una verificación que omite la regeneración si la colección ya contiene documentos.

IX. PLAN PARA LA SEGUNDA ENTREGA

La segunda entrega incorporará los componentes restantes del sistema. La Tabla II resume las tareas planificadas.

En particular, para los componentes más complejos se describen a continuación las decisiones de diseño tentativas:

TABLE II
PLAN DE TRABAJO PARA LA SEGUNDA ENTREGA

Componente	Estado objetivo
Agente Transaccional	Por implementar
MCP Server con herramientas controladas	Por implementar
Base de datos relacional ficticia en Docker	Por implementar
Reglas de seguridad y anonimización MCP	Por implementar
Memoria temporal de sesión	Por implementar
Memoria histórica persistente	Por implementar
Evaluación experimental (35 preguntas)	Por implementar
Corrección de observaciones de la primera entrega	Por implementar

A. MCP Server y base transaccional

Se construirá un servidor MCP usando FastAPI o la librería oficial de MCP para Python. La base de datos relacional se ejecutará en Docker (PostgreSQL) y será poblada con un script Python que genere clientes, cuentas, transacciones normales y transacciones sospechosas siguiendo los patrones definidos en el enunciado. Las herramientas expuestas serán: `get_transaction_by_id`, `search_transactions`, `get_customer_risk_summary`, `get_recent_flagged_transactions` y `create_fraud_case`. No se expondrá ninguna herramienta de ejecución SQL libre.

B. Memoria conversacional

La memoria temporal de sesión se implementará manteniendo el historial de mensajes en la sesión de Streamlit (`st.session_state`). La memoria histórica persistente se almacenará en la misma base de datos PostgreSQL, en una tabla de sesiones y mensajes, permitiendo consultas como “¿qué pregunté anteriormente sobre descenso del gradiente?”.

X. CONCLUSIONES

El prototipo de primera entrega demuestra que la arquitectura base del sistema multi-agente es funcional: el orquestador selecciona correctamente las herramientas, el pipeline RAG recupera fragmentos relevantes desde los apuntes, el agente web complementa la información cuando es necesario y Langfuse registra trazas completas de cada ejecución. Los principales desafíos identificados son la calidad variable de los PDFs fuente, la nomenclatura inconsistente de algunos archivos y los límites de la API gratuita de Tavily. Estos problemas serán abordados en la segunda entrega junto con la incorporación del MCP Server, la base de datos transaccional, la memoria persistente y la evaluación experimental completa.

REFERENCIAS

REFERENCES

- [1] OpenAI, “Function calling,” 2026. [Online]. Available: <https://platform.openai.com/docs/guides/function-calling>. [Accessed: Jun. 2026].
- [2] Chroma, “Chroma Documentation,” 2026. [Online]. Available: <https://docs.trychroma.com>. [Accessed: Jun. 2026].

- [3] Langfuse, "Langfuse Documentation," 2026. [Online]. Available: <https://langfuse.com/docs>. [Accessed: Jun. 2026].
- [4] Tavily, "Tavily Search API," 2026. [Online]. Available: <https://docs.tavily.com>. [Accessed: Jun. 2026].
- [5] LangChain, "Text Splitters," 2026. [Online]. Available: https://python.langchain.com/docs/concepts/text_splitters/. [Accessed: Jun. 2026].
- [6] Streamlit, "Streamlit Documentation," 2026. [Online]. Available: <https://docs.streamlit.io>. [Accessed: Jun. 2026].